SQL Review

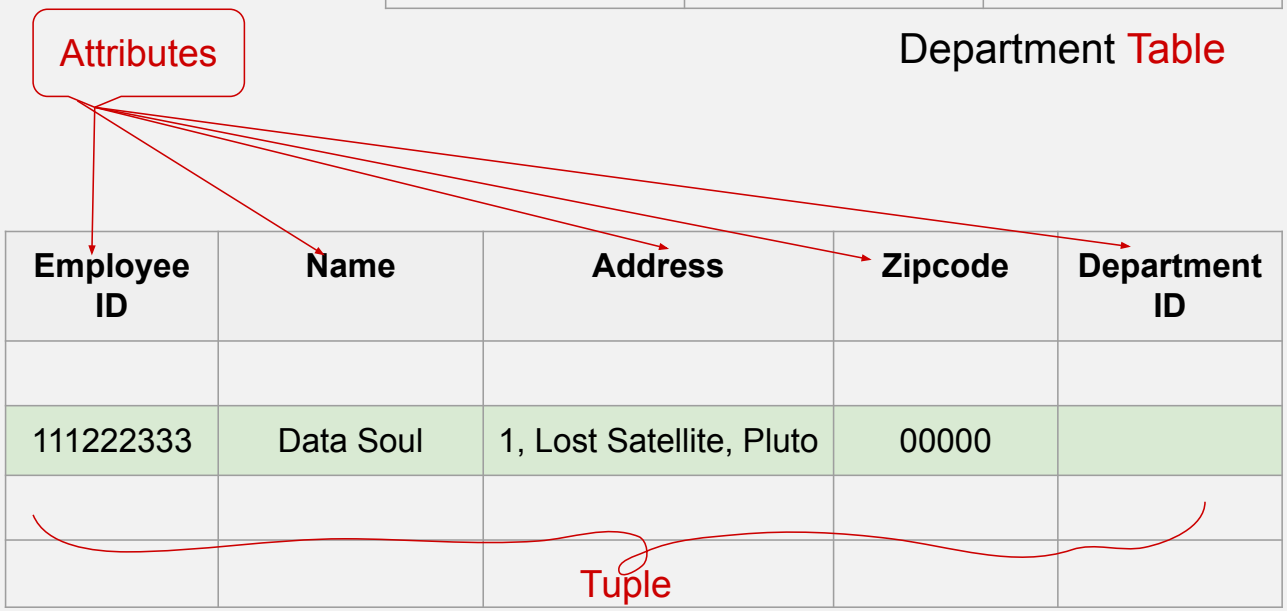
Relational Databases

- A relational database is a collection of data organized as a set of formally described tables from which data can be accessed or reassembled in many different ways without having to reorganize the database tables.
- A RDBMS (Relational Database Management System) is a software system that enables users to define, create, maintain, and control access to the database. Examples of RDBMS include Oracle, MySQL, PostgreSQL, Microsoft SQL Server, IBM DB2, and many others.

Relational Database

Department ID	Name	Location

Department Table



Employee Table

Relational database: A set of tables, defined by a schema

- Relational databases are based on the concept of relational algebra on sets.
- The terms table, relation, set are often used interchangeably.
- The terms row, tuple, record are often used interchangeably.

SQL

- SQL: Structured Query Language – the standard for manipulating relational databases. It is a declarative language that allows users to specify what they want without having to specify how to get it.
- Here is a good SQL tutorial.
  https://www.sqltutorial.org/
- SQL statements typically have three parts: Verb (Action), Subject (Target), and Condition (Filter).
- The Action is usually a *SELECT*, *INSERT*, *UPDATE*, or *DELETE* statement. The Subject is the table(s) on which the action is performed. The Condition is a filter that specifies which rows are affected by the action.

Relational Algebra Operations

Basic relational algebra operations that form the basis for relational databases.

- Selection
- Projection
- Union
- Intersection
- Difference
- GroupBy and Aggregation
- Join - Inner, Left (Right) Outer, Full Outer Join, Cross Join, Self Join

We show the corresponding SQL database queries.

Selection

The SELECT statement is used to retrieve data from one or more tables.

```
SELECT
  select_list
FROM
  table_name
```

Example:

```
select
from  Table_T
where (predicate is true);
```

The optional ORDER BY clause allows you to sort the result set by one or more sort expressions in ascending and/or descending order.

```
SELECT
  select_list
FROM
  table_name
ORDER BY
  sort_expression [ASC | DESC];
```

Projection

Projection takes a table (relation) and a (possibly empty) list of attributes of that table as input. It outputs a table containing only the specified list of attributes with duplicate tuples removed.

```
select  distinct A1, A2, A3  // where {A1, A2, A3} is a subset
from     Table_T;  // of the attributes for the table
```

Example:

```
SELECT DISTINCT
  salary
FROM
  employees
ORDER BY
  salary DESC;
```

WHERE Clause

To select specific rows from a table based on one or more conditions, you use the WHERE clause in the SELECT statement.

```
SELECT
    column1, column2, ...
FROM
    table_name
WHERE
    condition;
```

Example:

```
SELECT first_name, last_name, hire_date
FROM
  employees
WHERE
  hire_date >= '1999-01-01'
ORDER BY
  hire_date DESC;
```

Union

The UNION operator allows you to combine the result sets of two SELECT statements into a single result set.

```
SELECT
    column1, column2
FROM
    table1
UNION
SELECT
    column3, column4
FROM
    table2;
```

Intersection

The INTERSECT operator finds the common rows of the result sets of two SELECT statements.

```
SELECT
  column1,
  column2
FROM
  table1
INTERSECT
SELECT
  column1,
  column2
FROM
  table2;
```

Difference

The MINUS operator allows you to find the difference between two result sets of two SELECT statements.

```
SELECT
  column1,
  column2
FROM
  table1
MINUS
SELECT
  column1,
  column2
FROM
  table2;
```

The MINUS operator returns only the rows that appear in the result set of the first SELECT statement but not the second.

Group By and Aggregation

The GROUP BY clause allows you to group rows based on values of one or more columns. It returns one row for each group.

```
SELECT
  column1,
  column2,
  aggregate_function (column3)
FROM
  table_name
GROUP BY
  column1,
  column2;
```

It is often used with aggregate functions such as COUNT, SUM, AVG, MAX, and MIN to perform calculations on each group of rows.

```
SELECT fruit, COUNT(id)
FROM sample_table
GROUP BY fruit;
```

```
select pub_id, type, avg(price), sum(total_sales)
from titles
group by pub_id, type
```

Joins

- A *Join* is a means for combining fields from two tables by using values common to each. ANSI standard SQL specifies four types of JOIN: INNER, OUTER, LEFT, and RIGHT. As a special case, a table can JOIN to itself in a self-join.
- Inner Join: returns the intersecting rows from the two tables.
- Left Outer: returns all rows from the left table whether or not there is a matching row in the right table. (Similarly for a right outer join)
- Full Outer Join: returns all rows from the joined tables whether or not the other table has the matching row.
- Cross Join: returns the Cartesian product of two tables. For tables of size n and m, their result has $n \times m$ rows.
- Self Join: join a table to itself using either an inner join or left join clause

SQL also does an implicit join if we just use the WHERE clause on multiple tables – it defaults to a CROSS JOIN. This style of writing SQL is discouraged. Read about each join in detail in the SQL tutorial.

Join Example

```
CREATE TABLE department (
 DepartmentID INT,
 DepartmentName VARCHAR(20));

CREATE TABLE employee (
 LastName VARCHAR(20),
 DepartmentID INT);

SELECT *
FROM employee
INNER JOIN department ON
employee.DepartmentID = department.DepartmentID;

--implicit join
SELECT *
FROM employee, department
WHERE employee.DepartmentID = department.DepartmentID;
```

More on Joins

- Relational databases are often normalized to eliminate duplication of information when objects may have one-to-many relationships.
- Joining two tables effectively creates another table which combines information from both tables. This is at some expense in terms of the time it takes to compute the join.
- While it is also possible to simply maintain a denormalized table if speed is important, duplicate information may take extra space, and add the expense and complexity of maintaining data integrity if data that is duplicated later changes.
- Three fundamental algorithms for performing a join operation are: *nested loop join*, *sort-merge join* and *hash join*.

Foreign Keys

- A foreign key is a column or a set of columns in a table that references the primary key of another table.
- The table containing the foreign key is called the child table, and the table containing the primary key is called the parent table.
- Foreign keys are used to establish and enforce a link between the data in the two tables.
- Foreign keys help maintain referential integrity in the database by ensuring that the value in the foreign key column(s) of the child table matches a value in the primary key column(s) of the parent table.

Join Example

User Demographics.

- Let R represent a collection of user profiles, in which case the key value k could be interpreted as the primary key (e.g., user id).
  - The tuples might contain demographic information such as age, gender, income, etc.
- The other data set, L, might represent logs of online activity. Each tuple might correspond to a page view of a particular URL and may contain additional information such as time spent on the page, ad revenue generated, etc.
  - The key value k in these tuples could be interpreted as the foreign key that associates each individual page view with a user.
- Joining these two data sets would allow an analyst, for example, to break down online activity in terms of demographics.

RDBMS exercise



- Write SQL queries to create the schema for the example we just covered.
- Write a few sample SQL queries. For example:
  - Print the department table.
  - Print all the addresses found in the Employee table. Could there be any duplicates?
  - Print all (unique) zipcodes where employees live.
  - Print all the name and department IDs in the Employee table. Would they be unique?
  - Print the employee ID, their name and the name of the department where they work.

Join Examples



Movie Data Mining. Suppose we have three tables shown below and we want to find the top ten movies (by average rating):

```
Ratings [UserID, MovieID, Rating, Timestamp]
Users   [UserID, Gender, Age, Occupation, Zip-code]
Movies  [MovieID, Title, Genres]
```

Team up with another student and figure out the SQL query to get the answer.

Hint: https://www.sqltutorial.org/sql-aggregate-functions/sql-avg/